
SOFTWARE ENGINEERING

NOTEBOOK

Table of Contents

Table of Contents	2
1 Design Patterns	5
1 Creational Patterns	6
1.1 Factory Method	6
1.2 Abstract Factory	8
1.3 Builder	10
1.4 Prototype	12
1.5 Singleton	14
2 Behavioral Patterns	15
2.1 Chain of Responsibility	15
2.2 Command	17
2.3 Iterator	19
2.4 Mediator	21
2.5 Memento	23
2.6 Observer	24
2.7 State	26
2.8 Strategy	28
2.9 Template Method	30
2.10 Visitor	32
3 Structural Patterns	34
3.1 Adapter	34
3.2 Bridge	35
3.3 Composite	37
3.4 Decorator	39
3.5 Facade	41
3.6 Flyweight	42
3.7 Proxy	44
Design Patterns Bibliography	46
2 Python	47
1 PEP	47

2	Built-in Data Types	47
2.1	dict	48
2.2	list	48
2.3	set	48
2.4	frozenset	48
2.5	tuple	48
2.6	string	48
2.7	bytes	48
2.8	bytearray	48
3	unittest Module	49
3.1	Classes and functions	49
4	Decorator	50
5	Special Instance Method	51
6	Providing Multiple Ctors	51
	Python Bibliography	52

Design Patterns

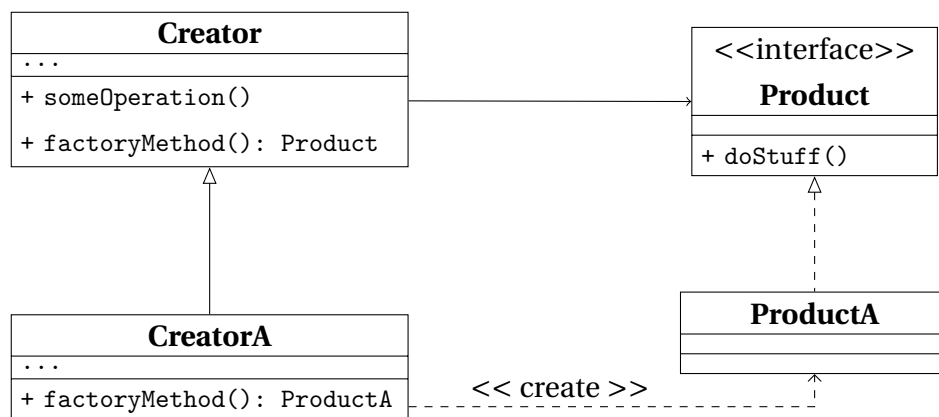
Design Patterns describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.

1 Creational Patterns

1.1 Factory Method

What for	<i>creating objects</i> without having to specify the exact class of the object that will be created
How to	a <i>factory method</i> is used <i>instead of calling a constructor</i> : it can be either <i>specified in an interface</i> and implemented by child classes, or <i>implemented in a base class</i> and optionally overridden by derived classes
When needed	• exact types and dependencies of the objects are unknown beforehand • internal components of a library or framework need to be extended by users • system resources need to be saved by reusing existing objects.

Structure



ABCProduct	interface common to all the objects.
ConcreteProduct	different implementations of the product interface.
Creator	base class which declares the factory method: • factory method - abstract or returning a default product; • returned type - must match the interface. It can have some <i>core business logic</i> which can be splitted from the product creation.
CreatorA, CreatorB	concrete classes that <i>override the base factory method</i> to return the different types of concrete products.

Table 1.1: Factory Method Class Diagram.

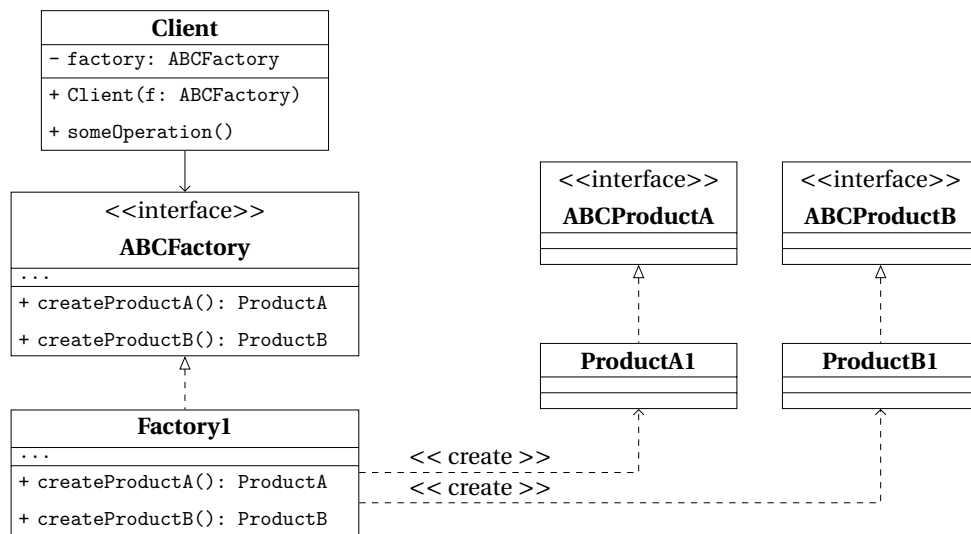
Pros and Cons

Pros	• <i>decoupling</i> of the creation of concrete objects from the core business logic of the Creator; • <i>Single Responsibility Principle</i> • <i>Open/Close Principle</i>
Cons	code gets more complex

1.2 Abstract Factory

What for	<i>producing families of related items</i> without specifying their concrete classes
How to	declaring <i>explicit interfaces</i> for each distinct product of the product family, and making all variants of products follow those interfaces
When needed	• client needs to work with various families of related objects, without depending on the latter's concrete classes • a set of <i>factory methods</i> blurring the main responsibility of a class needs to be managed better

Structure



ABCProductA, ABCProductB	interfaces for a set or family of related but distinct products
ProductA1, ProductB1	implementations of the concrete variants of abstract products. Each abstract product must be implemented in all given variants.
ABCFactory	interface for the sets of methods needed to create each of the abstract products
Factory1	implement the creation methods of the abstract factory, each creating only the corresponding product variant
Client	can work with any concrete factory as long as it communicates with the objects through their abstract interfaces. Therefore, the signature of concrete-factories' creation-methods must match the corresponding abstract interfaces'.

Table 1.2: Abstract Factory Class Diagram.

Pros and Cons

Pros	• <i>interchangeable concrete implementations</i> without changing the code that uses them • <i>no tight coupling</i> between concrete products and client code • <i>Single Responsibility Principle</i> • <i>Open/Close Principle</i>
Cons	• unnecessary complexity and work in the initial writing • systems can get more difficult to debug and maintain due to higher separation and abstraction

1.3 Builder

What for	<i>building complex objects step-by-step</i> , producing different types and representations using the same construction steps
How to	<i>object's construction code is moved into a Builder</i> class, that builds the object through steps that can be executed in series and only if needed. Several Builders can be implemented in case different ways of building make sense. A <i>Director</i> class can be used to manage the order of the building steps.
When needed	• ctors became telescopic because of too many optional parameters and need to be removed • code need to be able to create different representations of some products • a <i>composite</i> object construction needs to be managed step-by-step

Structure

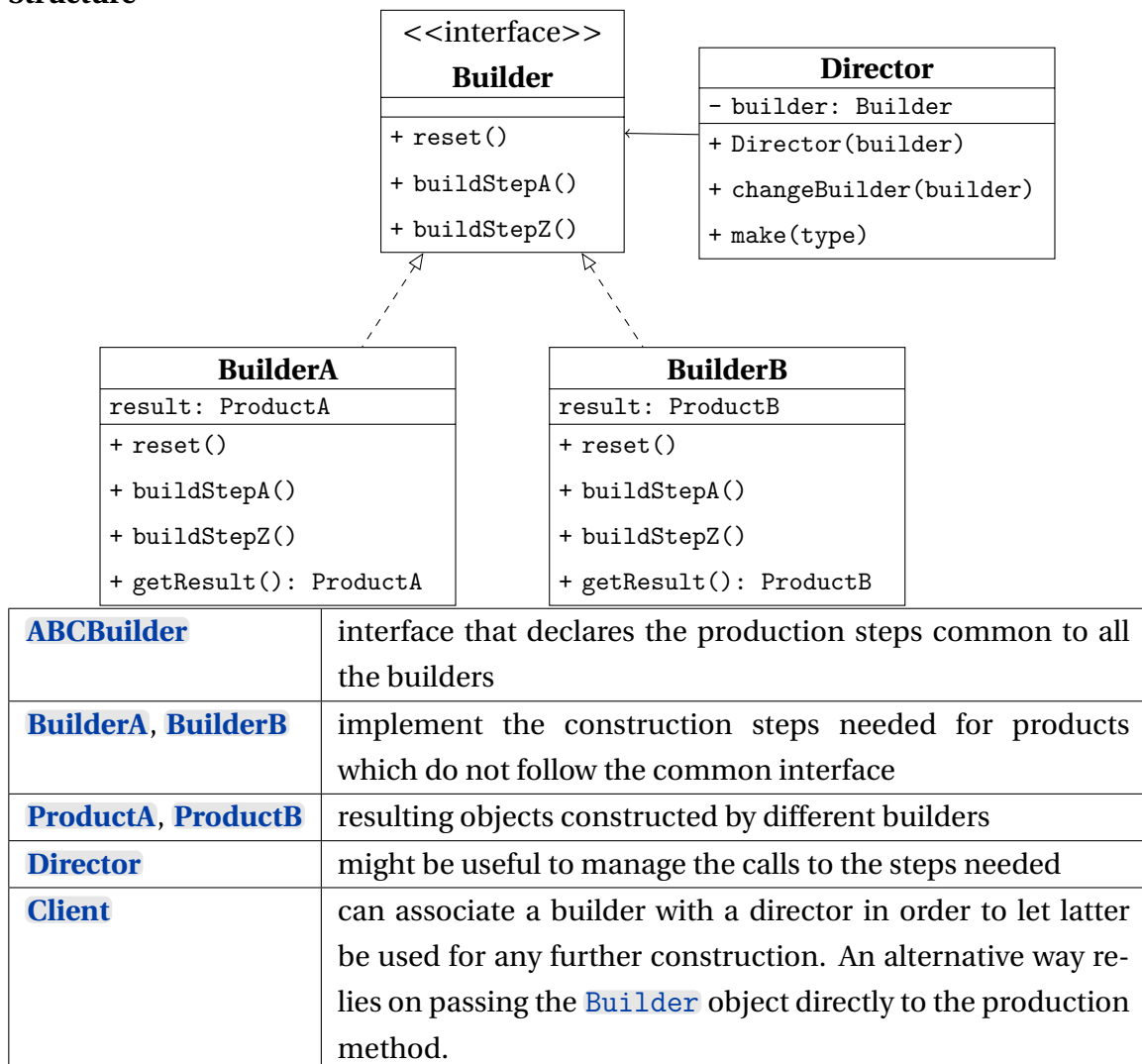


Table 1.3: *Builder Class Diagram.*

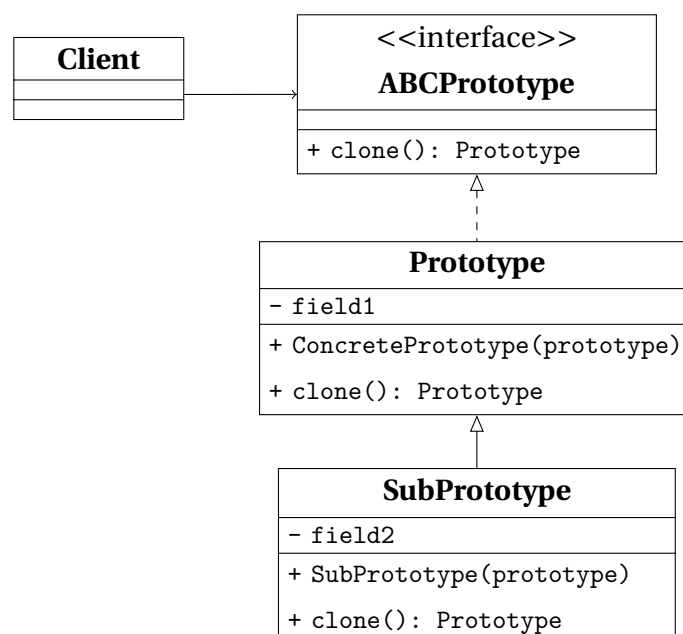
Pros and Cons

Pros	• construction goes step-by-step and can be deferred or run recursively • construction steps can be shared in case • Single Responsibility Principle
Cons	multiple classes required by this pattern can increase the code complexity

1.4 Prototype

What for	<i>copying existing objects</i> without implementing code which depends on their classes
How to	a common interface is implemented for all objects that need to be cloned, and the actual cloning procedure is delegated to the objects themselves in order to have access to private and protected details as well
When needed	- code shouldn't depend on the concrete classes of the objects being copied - reducing the number of subclasses only differing in the way of initializing their respective objects.

Structure



ABCPrototype	interface that declares the cloning method
Prototype	class that implements the cloning method; it can have some logic needed to manage edge cases
SubPrototype	a class that can be handy to add subsets of prototypes
Client	can produce a copy of any object which follows the prototype interface

Table 1.4: *Prototype Class Diagram.*

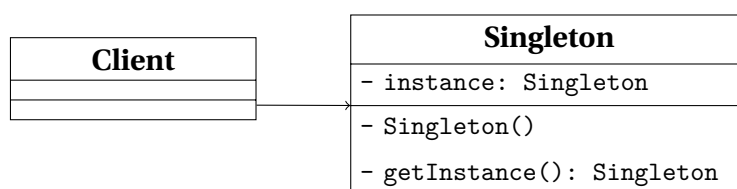
Pros and Cons

Pros	• cloning objects without coupling to their concrete classes • repeated initialization can get removed in favor of cloning pre-built prototypes • easier production of complex objects • alternative to inheritance when configuring complex objects
Cons	cloning can get more difficult when circular references are present

1.5 Singleton

What for	<i>ensuring that a class could have one instance only</i> , while providing a global access point to the instance
How to	the <i>ctor is made private</i> to prevent calls from outside the class and a <i>static creation method</i> is used in its place in order to create the object and return it for successive calls.
When needed	• a class in a program should have just a single instance available, as it can be needed for database object shared by different parts of the code • a stricter control over global variables is needed

Structure



Singleton	class declaring a static method that return the same instance it creates the first time it's called; this class' ctor is made private to hide it from external calls.
------------------	---

Table 1.5: *Singleton Class Diagram.*

Pros and Cons

Pros	• avoiding <i>global namespace pollution</i> introducing global variables • <i>initialization</i> takes place only the first time the object is requested • <i>multiple but limited instances</i> can be created, modifying only the static method
Cons	• violation of <i>Single Responsibility Principle</i> • can mask bad design decisions, like too much overlapping between components • multithreaded programs need special care to avoid multiple instances creation • unit tests could be difficult due to the impossibility for mock objects to access the Singleton's ctor.

2 Behavioral Patterns

2.1 Chain of Responsibility

What for	<i>pass a request along a chain of handlers</i> , each of which can either manage the request or pass it to the next handler in the chain
How to	each particular behavior is implemented into a stand-alone handler, the handlers are linked one to another in a chain and the request, along with the data needed, is passed along the chain
When needed	- different kind of requests are expected to be processed in different ways and the type of the request is not known beforehand - the set or the order of the handlers can change at run time

Structure

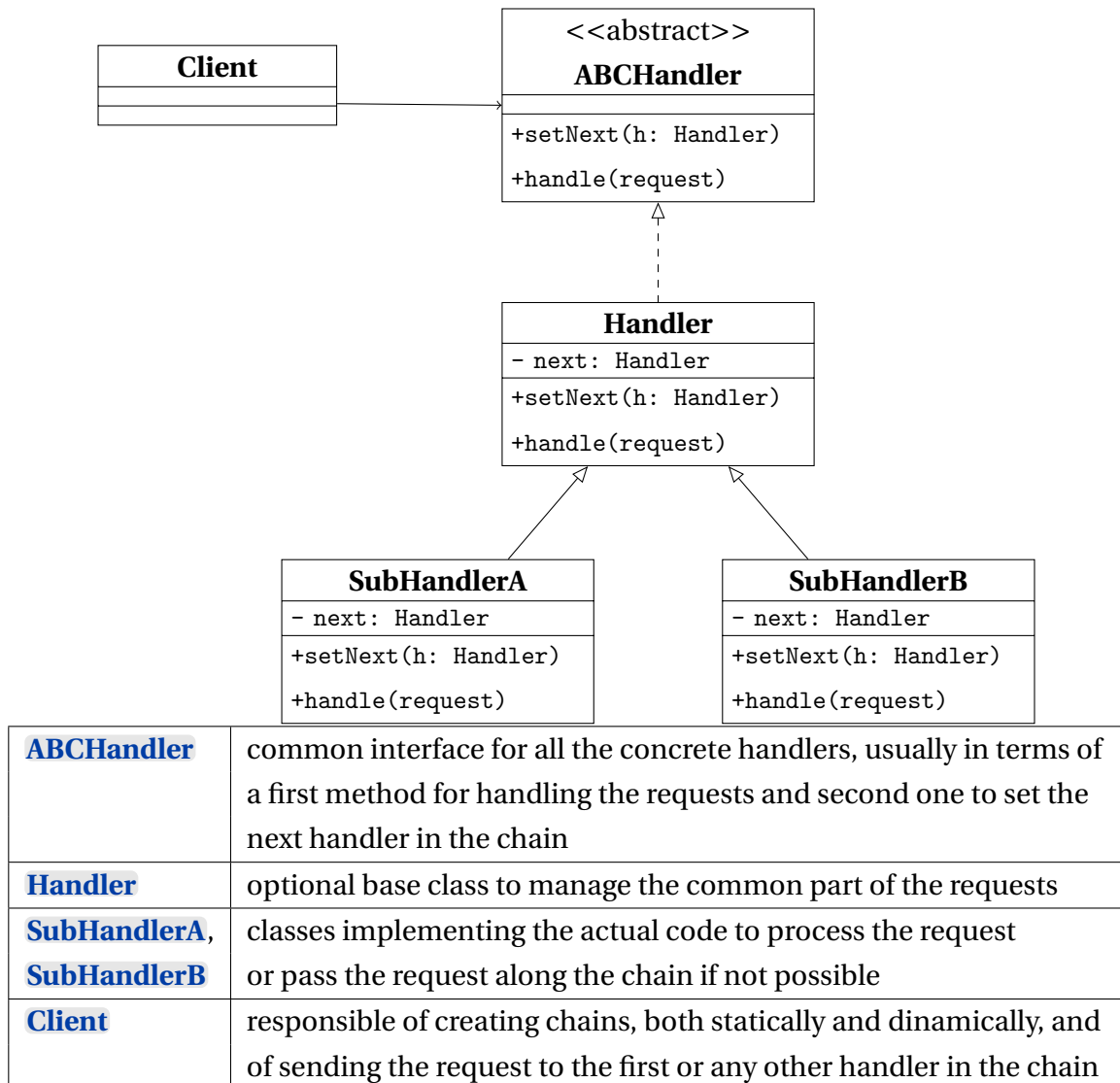


Table 1.6: Chain of Responsibility Class Diagram.

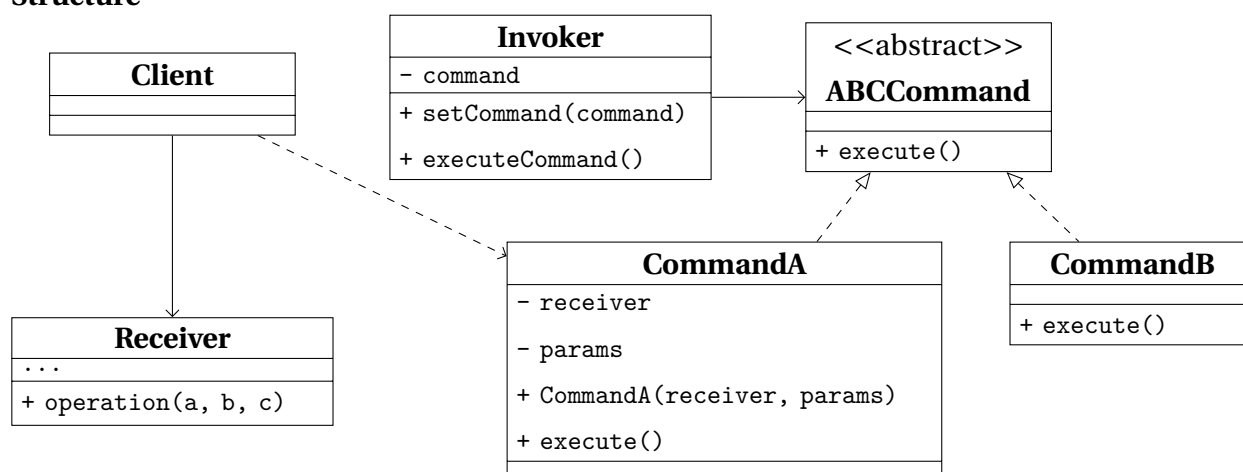
Pros and Cons

Pros	• the order of request handling can be under complete control • flexibility in deciding whether a handler should stop the request or pass it along the chain • <i>Single Responsibility Principle</i> • <i>Open/Close Principle</i>
Cons	some requests may end up unhandled

2.2 Command

What for	<i>turning a request into a stand-alone object</i> containing all information about it.
How to	applying the <i>Principle of Separation of Concerns</i>
When needed	• there is the need to parametrize objects with operations • there is the need for reversible operations

Structure



Invoker / Sender	class responsible for creating and sending requests, directly triggering a reference to a command object, usually provided and not created by the sender
ABCCommand	interface declaring the method to execute the command
CommandA, CommandB	classes implementing the concrete requests, passin the call to one of the business logic objects
Receiver	class that implements the business logic and does the actual work most of the times
Client	class responsible for creating and configuring commands, passing all the parameters needed

Table 1.7: *Command Class Diagram.*

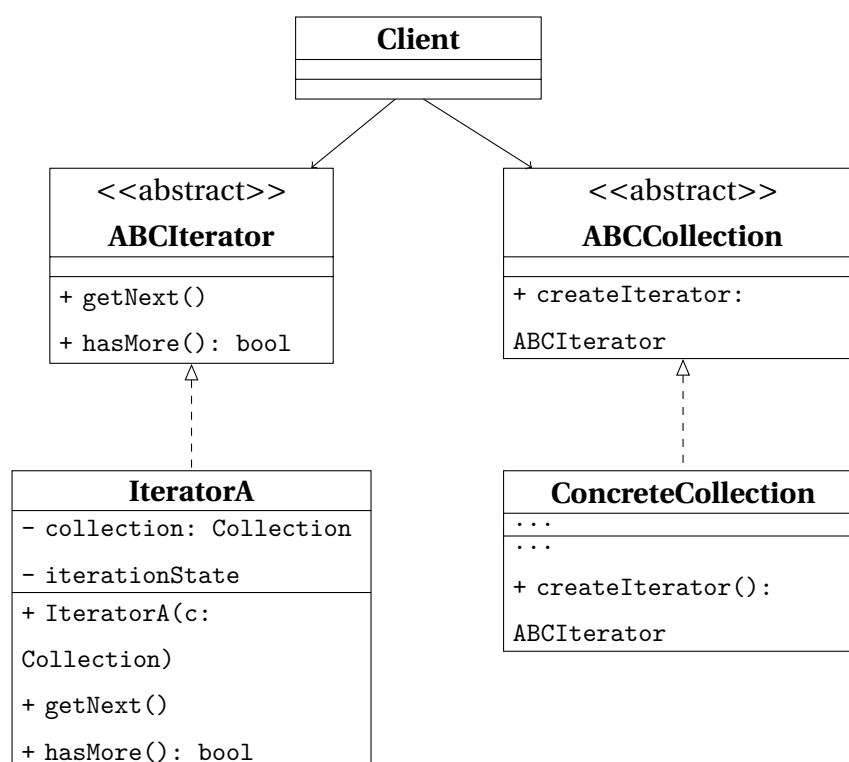
Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Close Principle</i> • implementing <i>undo/redo</i> is easier • implementing deferred execution of operation is easier • set of simple commands can be assebled in sets
Cons	programs end up having more layers

2.3 Iterator

What for	<i>traversing elements of a collection without exposing its underlying representation</i>
How to	encapsulating the details of the traversal into a object called iterator
When needed	• collection with complex data structure not to expose to users • reducing code duplication for traversals • traversing data structures, that can be unknown beforehand

Structure



ABCIterator	interface declaring the operations needed to traverse a collection, like fetching the next element, keeping track of the current position and so on
IteratorA	implements specific algorithms, independent from each others in order to be used in any number on the same collection
ABCCollection	interface declaring one or multiple methods for getting iterator compatible with the collection. The return type of these methods must match the Iterator interface.
CollectionA	returns instances of a particular concrete iterator class on client's request.
Client	can work with both the collection and the iterator using their interfaces.

Table 1.8: Iterator Class Diagram.

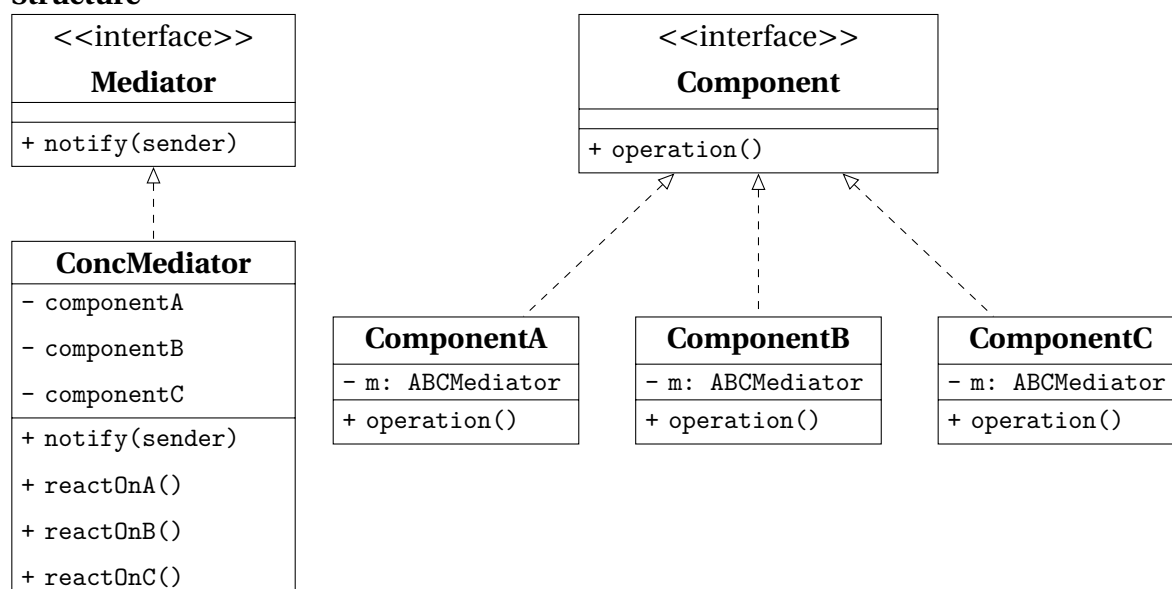
Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Close Principle</i> • parallel iterations are possible • iteration can be delayed to when it is needed
Cons	can be an overkill for simple collections can be less efficient than going directly through the elements of some specialized collection

2.4 Mediator

What for	<i>reducing chaotic and confusing dependencies</i> between objects and <i>restricting direct communications</i> between the objects
How to	
When needed	• changing an already existing class which is strongly coupled • reusing a strongly dependent component in a different program • reusing basic behaviours in completely different contexts

Structure



Component	interface for communication with other Components through its Mediator
ComponentA , ..., ComponentC	implements the Colleague interface and communicates with other Components through its Mediator
Mediator	interface for communication between Component objects
ConcMediator	implements the mediator interface, coordinating communication between Component objects. It is aware of all of the Component and their purposes with regards to inter-communication

Table 1.9: Mediator Class Diagram.

Pros and Cons

Pros	<i>Single Responsibility Principle</i> <i>Open/Close Principle</i> reducing coupling between various components reusing individual components is easier
Cons	mediator can evolve into a God object

2.5 Memento

What for	<i>saving/restoring states</i> of an object without exposing internals
How to	
When needed	• snapshots of an object's state are needed in order to restore them later • when encapsulation would be violated by directly accessing object's fields

Structure

Originator	Memento	Caretaker
- state	- state	- originator
+ save(): Memento	- Memento(state)	-history: Memento
+ restore(m : Memento)	- getState()	+ doSomething()
		+ undo()

Originator	generic class that can produce snapshots of its own state, and restore its state from one of them
Memento	class that acts like a shapshot, and could be made immutable
Caretaker	knows the <i>why</i> and <i>when</i> to capture a snapshot or restore a previous one. This class can manage an history of Originator's' states in a stack data structure.

Table 1.10: *Memento Class Diagram.*

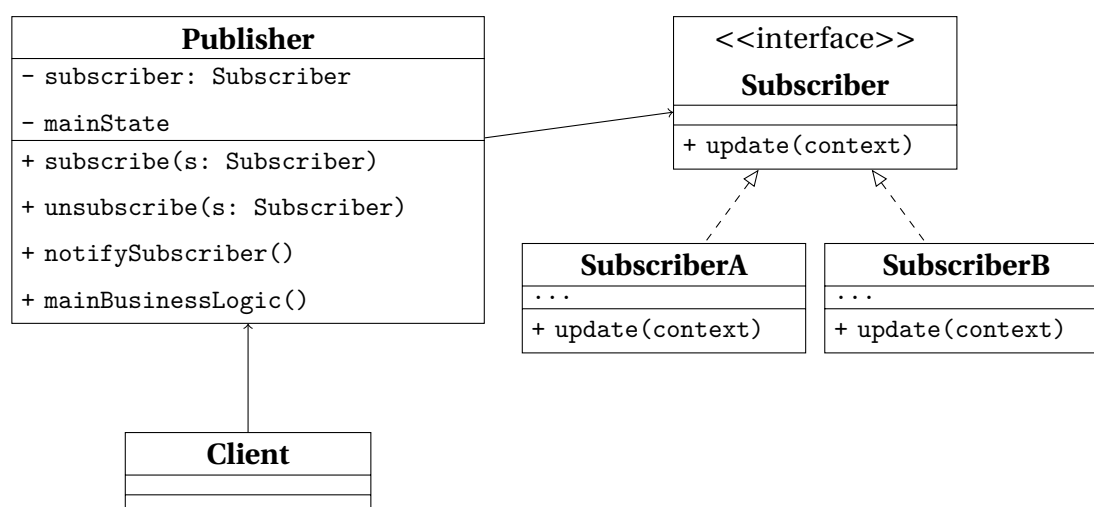
Pros and Cons

Pros	• producing snapshots of the object's state
Cons	• memory compsunction increases • to destroy outdated mementos, the originator's lifecycle should be tracked • modern dynamic languages do not guarantee the state within the memento stays untouched

2.6 Observer

What for	implementing a <i>subscription mechanism</i>
How to	
When needed	• changes to the state of one object may require changing other objects that are unknown beforehand or change dynamically • objects must observe each others, for a limited time and in specific circumstances

Structure



Publisher	class that issues events of interests to other objects. This class contains a subscription infrastructure that lets subscribers both join and leave the subscriber list
Subscriber	interface declares the notification interface, that in most cases is a single update method with some contextual parameters.
SubscriberA, SubscriberB	implement the different logics in response to notifications issued by the publisher.
Client	creates both publisher and subscriber objects and manage the registrations for updates

Table 1.11: Observer Class Diagram.

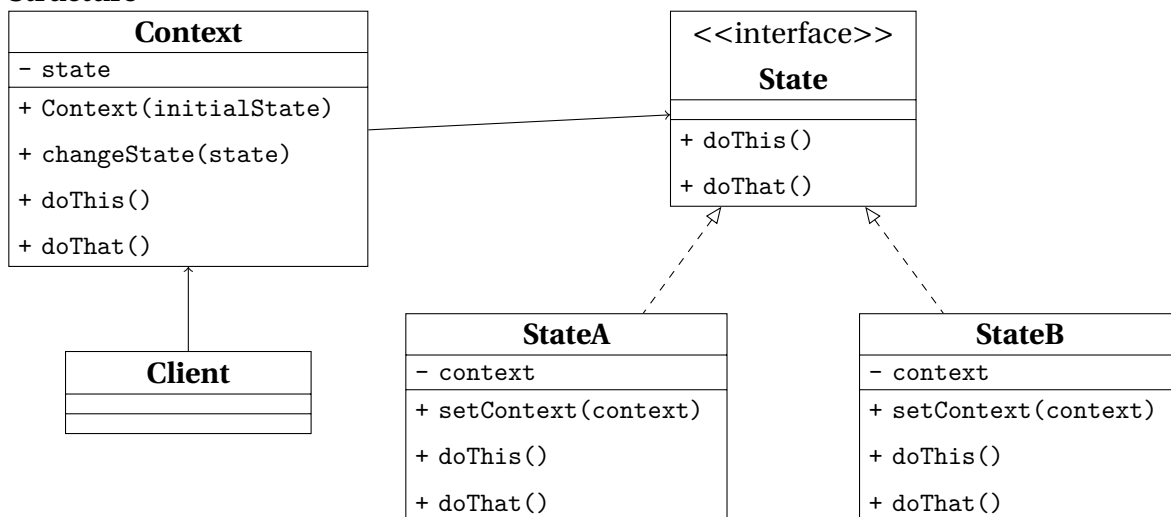
Pros and Cons

Pros	• Open/Close Principle • relations between objects can be established at runtime
Cons	• subscribers can be notified in random order

2.7 State

What for	implementing <i>objects whose behavior depends on their internal state</i>
How to	
When needed	• implementing an object that behaves differently depending on its current state, the number of states is considerable, and the state-specific code changes frequently • managing better a class that changes its behavior depending on some internal field actual value • managing better duplicate code across similar states and transition of a condition-based state machine

Structure



Context	stores a reference to one of the concrete state objects, to which it delegates all the state-related work. The Context interacts with the state interface, in order to be able to interact with any Concrete State
State	interface declares the state generic methods, that should make sense for all concrete states for a matter of consistency
StateA StateB	implement their own specific methods, and can make use of abstract classes to encapsulate some common behavior

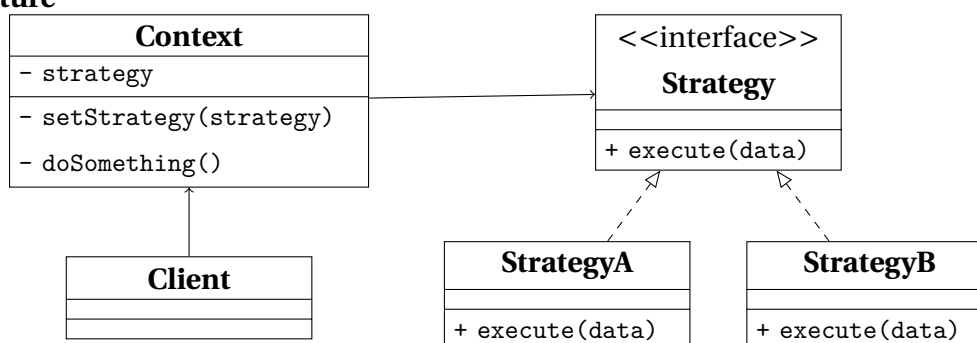
Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Close Principle</i> • bulky state machine code is eliminated
Cons	can be an overkill in case the state machine has a few states that change sporadically

2.8 Strategy

What for	defining a <i>family of algorithms, each implemented in a seprate class</i> , in oder to use them interchangeably
How to	
When needed	• different variants of the same algorithm must be used by an external object that can be able to switch between then • managing better lot of similar classes only differing in the way they execute some behavior • isolating business logic from the implementation details • better management of massive conditional statements that switches between variants of the same algorithm

Structure



Context	has a reference to the Strategy Interface, used to reference the Concrete Strategy in use, and methods to replace the strategy at run-time.
Strategy	interface define the part which is common to all concrete strategies and declares a method used by the Context to execute it.
StrategyA, StrategyB	implement a variety of algorithms needed by the context.
Client	manages the creation of the strategies and pass them to the context.

Table 1.12: Strategy Class Diagram.

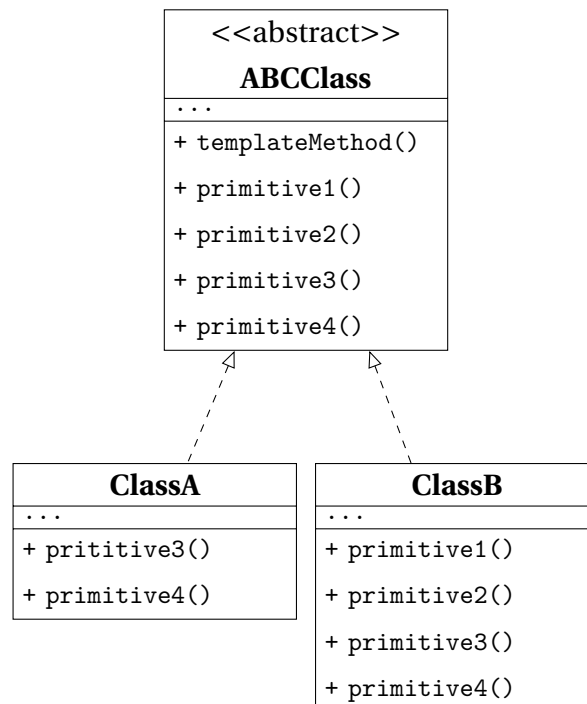
Pros and Cons

Pros	• swapping algorithms at runtime • implementation details of an algorithm is separate from the client using it • inheritance is replaced with composition • Open/Close Principle
Cons	• can be an overkill in case not many algorithms will be used and they will not change much at runtime • clients must know the differences between the algorithms to be able to choose • most of the modern programming languages implement different versions of an algorithm inside a set of anonymous functions and let the client use them in a similar way

2.9 Template Method

What for	<i>defining the skeleton of an algorithm in a class</i> and letting subclasses override specific steps
How to	based on <i>inheritance</i>
When needed	• client needs to extend only specific steps of an algorithm • managing better several classes containing almost identical algorithms

Structure



ABCClass	declares the steps of an algorithm and the <code>templateMethod</code> used to call them in a specific order. The steps may both be declared as abstract or implement some actual routines.
ClassA, ClassB	override all of the steps, but not the <code>templateMethod</code> itself.

Table 1.13: *Strategy Class Diagram.*

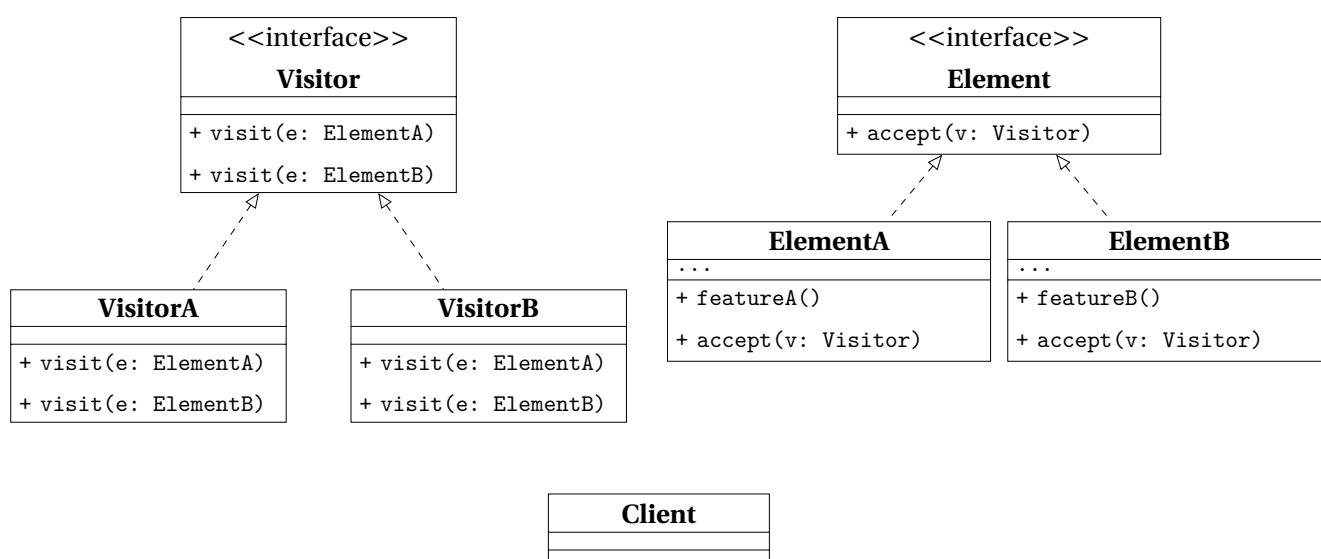
Pros and Cons

Pros	• overriding part of algorithms without affecting the rest • less code duplication
Cons	• limitations due to the skeleton of the algorithm • <i>Liskov Substitution Principle</i> • maintainance can get harder as the steps increase

2.10 Visitor

What for	<i>separating algorithms from the objects</i> they operate on
How to	
When needed	- performing an operation on all elements of a complex data structure - cleaning up the business logic extracting other behaviors - separating behaviors that make sense only for specific classes

Structure



Visitor	interface declares a set of visiting methods, each with a different Concrete Element taken as argument.
VisitorA, VisitorB	implement the different behaviors for each Concrete Element.
Element	interface declares a method to accept the Visitor interface as argument.
ElementA, ElementB	implements the acceptance method, whose purpose is the redirection the call to the proper visitor's method for the current element class. This method must be overridden in any subclass even though it was already developed in the base one.
Client	usually needs to work on a collection or a similar complex object whose type is ignored by the Client, since it works with the abstract interface.

Table 1.14: Strategy Class Diagram.

Pros and Cons

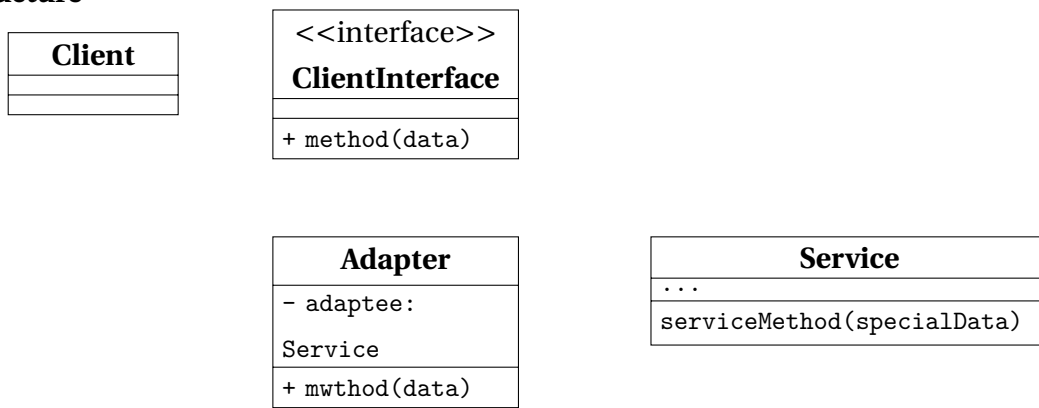
Pros	• <i>Open/Close Principle</i> • <i>Single Responsibility Principle</i> • can store information while working with different objects
Cons	• adding/removing a class requires all visitors to get updated • access grants to private data and methods of objects they are working on

3 Structural Patterns

3.1 Adapter

What for	allowing <i>objects with incompatible interfaces to collaborate</i>
How to	
When needed	• a class' interface is incompatible with the rest of the code • reusing subclasses that lack some common functionalities that can't be added in the superclass

Structure



Client	the business logic of the program
ClientInterface	the protocol for a class to be able to collaborate with it.
Service	usually a 3rd party or legacy software with an interface incopatible with the Client.
Adapter	a class able to work with both the client and the service. It receives calls from the client and translates into calls to a wrapped service object.

Table 1.15: *Strategy Class Diagram.*

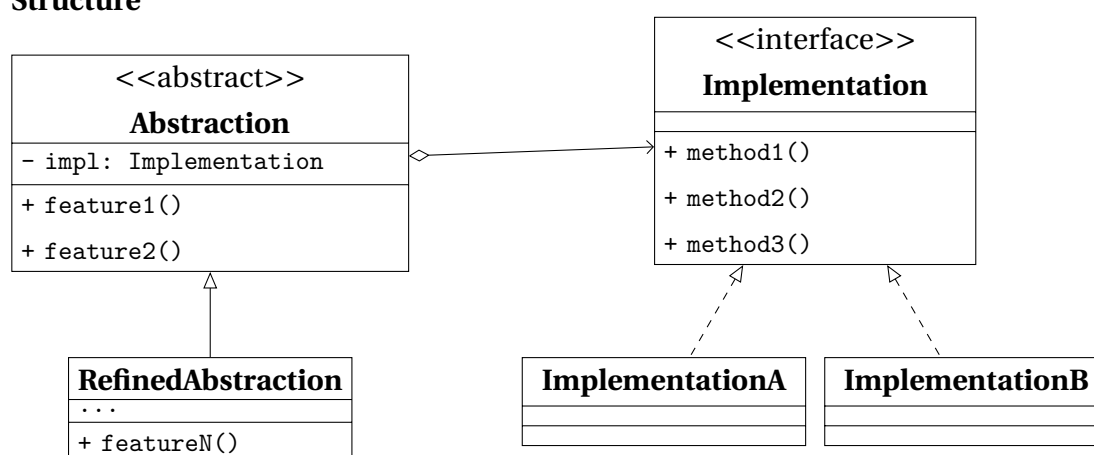
Pros and Cons

Pros	• <i>Single Responsibility Principle</i> • <i>Open/Close Principle</i>
Cons	• sometimes it's simpler to change the service class in order to match than adding a new set of interfaces

3.2 Bridge

What for	<i>splitting a large class or a set of related classes into two hierarchies</i> , an abstraction and an implementation, developed independently from each other
How to	
When needed	• dividing and organizing a monolithic class that has several variants of some functionality • extending a class in several independent dimensions • switching implementation at runtime is a requirement

Structure



Abstraction	high-level interface, and relies on the Implementation objects to perform the actual work.
RefinedAbstraction	optional and might help providing variants of control logic.
Implementation	interface shared by all the concrete implementations. Usually the abstraction declares some complex behavior that relies on primitive operations declared by the implementation.
ImplementationA, ImplementationB	implement specific code
Client	interacts with the abstraction, after linking it with one of the implementation objects.

Table 1.16: Bridge Class Diagram.

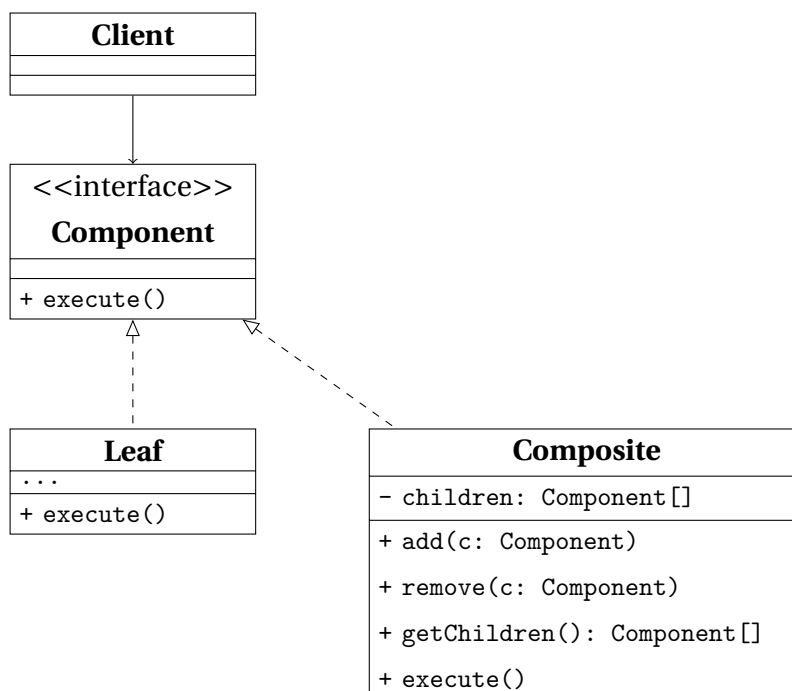
Pros and Cons

Pros	• platform-independent classes are easier to create • client isn't exposed to platform details and work with high-level abstractions • Open/Close Principle • Single Responsibility Principle
Cons	an highly cohesive class might require code getting more complicated

3.3 Composite

What for	<i>composing objects into tree structures</i> in order to work with these structures as if they were individual objects
How to	both leaf and composite objects implement the same component interface, the former working as a single entity and the latter possibly containing sub-elements, routines to add and remove them, and a routine to delegate work to them
When needed	• implementing a tree-like data structures • clients need to interact uniformly with simple and complex objects

Structure



Component	interface contains the common operations for both simple and complex elements of the tree.
Leaf	a basic element of a tree and doesn't contain any other sub-element. Usually is the part of the system which ends up doing the work.
Composite	an element that has sub-elements, which means leafs and containers as well. It interacts with sub-elements through their interfaces.
Client	can work with both simple and complex elements through the component interface.

Table 1.17: *Composite Class Diagram.*

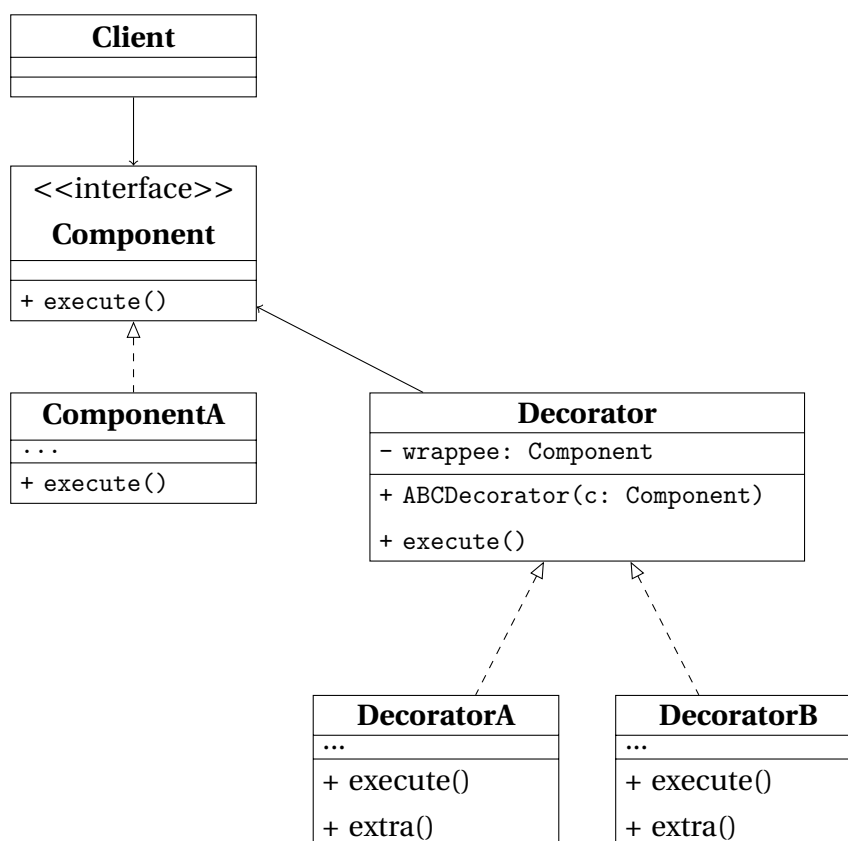
Pros and Cons

Pros	• polymorphism and recursion help work easier with complex structures • Open/Close Principle
Cons	a common interface can be difficult when functionalities of single elements differ too much from the ones of composite ones

3.4 Decorator

What for	<i>attach new behaviors to objects</i> , placing them in special wrappers
How to	objects are placed inside special wrapper objects that contain the behaviors
When needed	• extra behavior needs to be assigned to objects at runtime • extending object's behavior when inheritance can not be used (for example when a class is declared as <code>final</code>)

Structure



Component	interface common for both wrapper and wrapped objects.
ComponentA	class being wrapped and whose basic behavior can be altered by decorators.
Decorator	has a reference of the wrapped object, whose referenced type matches the Component interface. The Base Decorator delegates all operations to the wrapped object.
DecoratorA, DecoratorB	define behaviors which can be added to Components dynamically. The concrete decorator overrides the base behavior and execute the variant either before of after calling the parent method.

Table 1.18: *Decorator Class Diagram.*

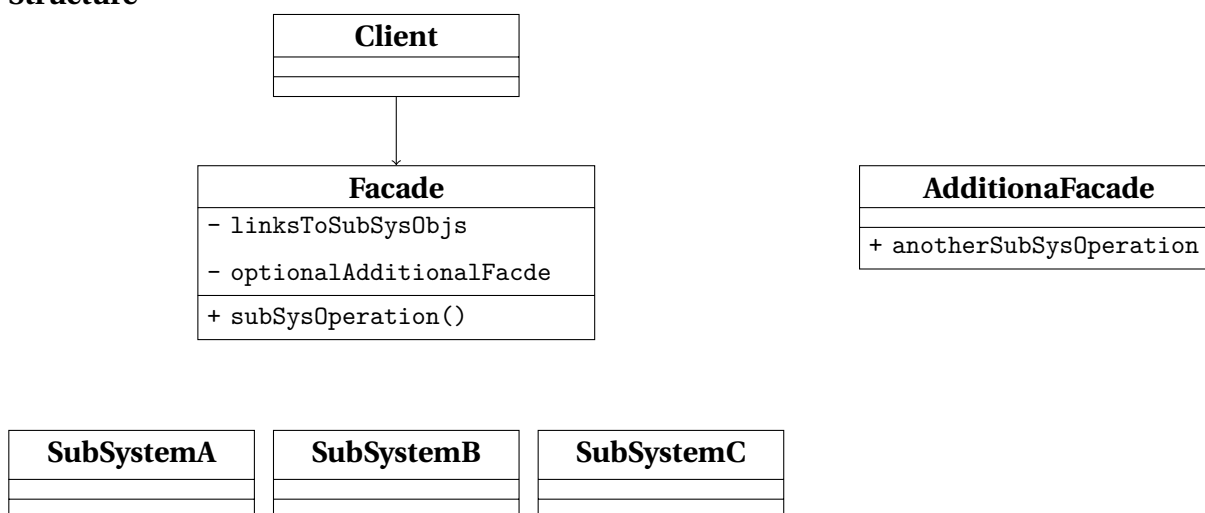
Pros and Cons

Pros	no other classes are needed to extend an object's behavior adding and removing responsibilities can be achieved at runtime ??
Cons	• removing wrappers from the stack can be difficult • implementing a decorator in a way its behavior does not depend on the order in the stack can be hard • code can get ugly when the layers increase in number

3.5 Facade

What for	providing users with a <i>simplified interface to any complex set of classes</i>
How to	a facade class provides a simple interface to a complex subsystem
When needed	• limited but straightforward interface to a complex subsystem is needed • subsystem needs to be structured into layers

Structure



Facade	implements a convenient access to a particular part of the subsystem's functionality and knows where to direct the client's request and how to operate all the system's parts.
AdditionalFacade	might help reducing the tasks managed by a single Facade. It can be used by the Client and by the other facades as well.
SubSystemA , ..., SubSystemC	Subsystem class are various object which need to be studied deeply in order to manage them properly. These classes work together and are unaware of the Facade's existence.
Client	calls the subsystem classes through the Facade only

Table 1.19: *Facade Class Diagram.*

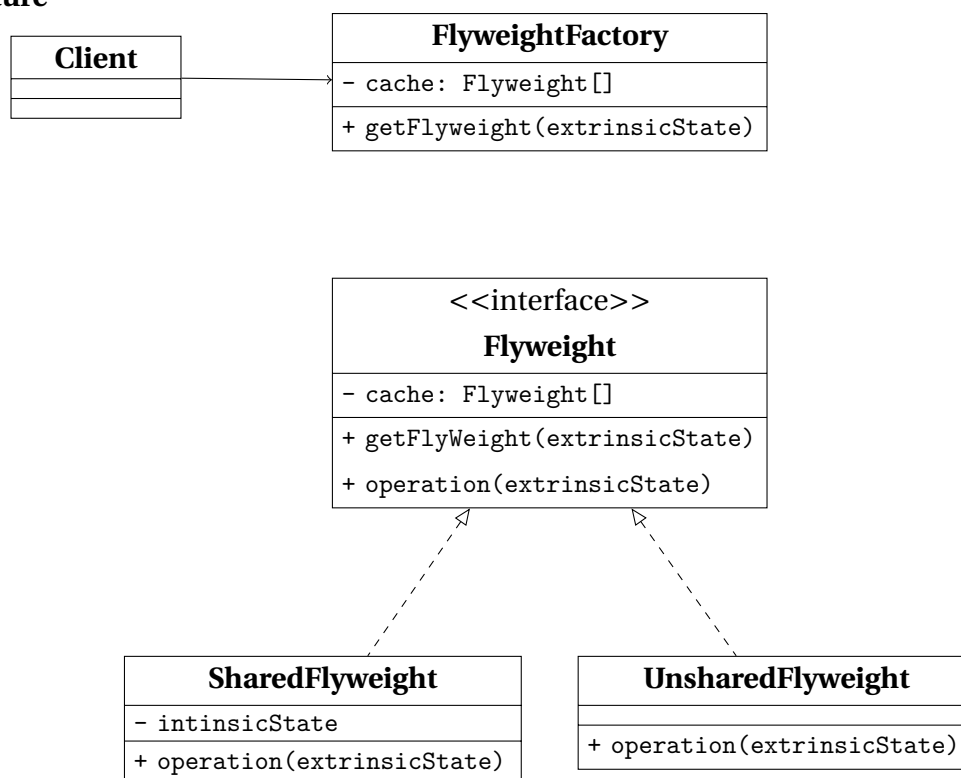
Pros and Cons

Pros	client code is isolated from the complexity of a subsystem
Cons	desire to couple a class to a facade can overwhelm the code base

3.6 Flyweight

What for	reducing RAM consumption by <i>sharing common parts of state between multiple objects</i> , instead of creating copies of same data in each object
How to	after extracting the <i>intrinsic state</i> (invariant, context-independent and shareable) an interface to the <i>extrinsic state</i> (variant, context-dependent and unshareable) is implemented
When needed	dealing with large numbers of objects with simple repeated elements that would use a large amount of memory if individually stored

Structure



ABCFlyweight	
SharedFlyweight	
UnsharedFlyweight	
FlyweightFactory	a class to generate and share a pool of flyweight objects
Client	

Table 1.20: *Flyweight Class Diagram.*

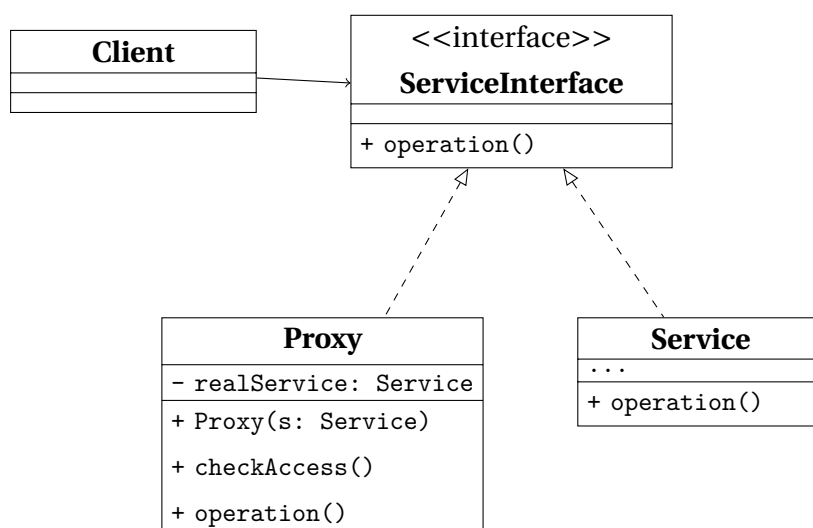
Pros and Cons

Pros	saving RAM while managing several similar objects
Cons	• CPU cycles might increase while RAM consumption decreases • code complexity and more overhead to understand the separation of states

3.7 Proxy

What for	<i>substitute or placeholder for another object</i> , <i>controlling access</i> to it giving the opportunity to make actions both before and after using the object
How to	a proxy class with the same interface of the original object is responsible of accepting request and creating real service objects
When needed	• lazy initialization • access control • local execution of a remote service • logging the requests history through a proxy • caching requests result, especially useful when results are large • using references to keep track of the clients using a service, in order to dismiss it when not needed

Structure



ABCServiceInterface	the interface of the service which requires a proxy, that must follow this interface to disguise itself as the service.
Service	class providing some useful business logic.
Proxy	class containing a reference to a service object, to which it passes the request after some preliminary processing and of which it manages the lifecycle.
Client	works with both the service and the proxy, if they follow the same interface.

Table 1.21: Proxy Class Diagram.

Pros and Cons

Pros	• control over service objects without any knowledge about their internals • lifecycle of the service object does not require management by the client • proxy can work while the service is initializing making programs faster • <i>Open/Close Principle</i> respected since new proxies can be added without changing the service or its clients
Cons	• new classes make code more complex • responses from service can be delayed

Design Patterns Bibliography

- [147] *Design Patterns*. [Refactoring Guru](#).
- [148] *Factory Method Design Pattern*. [Wikipedia](#).
- [149] *Abstract Factory Design Pattern*. [Wikipedia](#).
- [150] *Builder Design Pattern*. [Wikipedia](#).
- [151] *Prototype Design Pattern*. [Wikipedia](#).
- [152] *Singleton Design Pattern*. [Wikipedia](#).
- [153] *Chain of Responsibility Design Pattern*. [Wikipedia](#).
- [154] *Command Design Pattern*. [Wikipedia](#).
- [155] *Iterator Design Pattern*. [Wikipedia](#).
- [156] *Mediator Design Pattern*. [Wikipedia](#).
- [157] *Memento Design Pattern*. [Wikipedia](#).
- [158] *Observer Design Pattern*. [Wikipedia](#).
- [159] *State Design Pattern*. [Wikipedia](#).
- [160] *Strategy Design Pattern*. [Wikipedia](#).
- [161] *Template Method Design Pattern*. [Wikipedia](#).
- [162] *Visitor Design Pattern*. [Wikipedia](#).
- [163] *Adapter Design Pattern*. [Wikipedia](#).
- [164] *Bridge Design Pattern*. [Wikipedia](#).
- [165] *Composite Design Pattern*. [Wikipedia](#).
- [166] *Decorator Design Pattern*. [Wikipedia](#).
- [167] *Facade Design Pattern*. [Wikipedia](#).
- [168] *Flyweight Design Pattern*. [Wikipedia](#).
- [169] *Proxy Design Pattern*. [Wikipedia](#).

1 PEP

2 Built-in Data Types

Python has several [types built in the interpreter](#); the principal ones are numerics, sequences, mappings, classes, instances and exceptions.

2.1 dict

2.2 list

2.3 set

2.4 frozenset

2.5 tuple

2.6 string

2.7 bytes

2.8 bytearray

3 unittest Module

3.1 Classes and functions

4 Decorator

A function decorator is a callable object that takes a function as its input parameter
[decorator](#)

[Decorators with parameters](#)

[PythonDecoratorLibrary](#)

5 Special Instance Method

Special Instance Method

6 Providing Multiple Ctors

multiple `__init__`

Python Bibliography

- [121] *Python Glossary*. docs.python.org.
- [122] *Providing Multiple Constructors in Your Python Classes*. [Real Python](#).
- [123] *Sorting Mini How To*. python.org.